



Ain Shams University – Faculty of Engineering I-CHEP
CSE356: Internet of Things

Project Report

Smart Mosque Management System

14-May-2026

Submitted to Prof. Islam Tharwat
Eng. Hamdy Osama

Submitted by 23P0184 – Sara Ahmed
23P0360 – El-Sharqawy Wael
23P0386 – Esraa Mustafa
23P0272 – Nada Medhat

Contents

1	Purpose & Requirements	3
1.1	Purpose of the IoT System	3
1.2	Functional Requirements	3
1.3	Non-Functional Requirements	4
1.4	Constraints and Assumptions	5
2	Process Specification	5
2.1	Overview	5
2.2	Core System Processes	5
2.2.1	Occupancy Detection Process	5
2.2.2	Energy Management Process	6
2.2.3	Wudu Water Monitoring Process	6
2.2.4	Air Quality Monitoring Process	6
2.2.5	Central Monitoring and Control Process	7
2.3	Integrated System Flowchart	7
2.4	Process Interaction Notes	7
3	Domain Model Specification	8
3.1	Entities	8
3.2	Attributes for each Entity	8
3.3	Relationships	10
3.4	UML Diagram	10
4	Information Model Specification	11
4.1	Types of Data in System	11
4.1.1	Sensor Data (Raw Data)	11
4.1.2	Processed Data	11
4.1.3	System Data (Stored)	11
4.1.4	Control Data (from Admin)	11
4.2	Data Flow	12
4.2.1	Main Flow	12
4.2.2	Control Flow	12
5	Service Specifications	13
5.1	Overview	13
5.2	Service List	13
5.3	Service Descriptions	13
6	IoT Level Specification	17
6.1	Level 1 – Perception Layer	18
6.2	Level 2 – Network Layer	19
6.3	Level 3 – Edge Processing Layer	19
6.4	Level 4 – Data Accumulation Layer	20
6.5	Level 5 – Application Layer	20
6.6	Level Interaction Summary	20
7	Functional View Specification	21
7.1	Overview	21
7.2	Functional Block Descriptions	21
7.3	Inter-Block Data Flow Summary	23

8	Operational View Specification	24
8.1	System Performance	24
8.1.1	Response Time	24
8.1.2	Data Update Frequency	24
8.1.3	Throughput	24
8.1.4	Efficiency	25
8.2	System Reliability	25
8.2.1	Sensor Failure Handling	25
8.2.2	Network Failure	25
8.2.3	Power Failure	25
8.3	System Scalability	25
8.3.1	More Sensors	25
8.3.2	Spreading to Other Mosques	25
8.3.3	Modular Architecture	25
8.4	System Security	26
8.4.1	Network Protection	26
8.4.2	Access Control	27
8.4.3	Data Encryption	27
8.4.4	Device Filtering	27
8.4.5	Privacy Protection	28
8.5	Energy Efficiency	28
8.5.1	Prayer Time Control	28
8.5.2	Occupancy-Based Activation	28
8.5.3	Automatic Shutdown	28
8.6	Safety Considerations	28
8.6.1	Device Overheating	29
8.6.2	Water Valve Reliability	29
9	Device and Component Integration	30
9.1	Input Devices (Sensors)	30
9.1.1	Motion Sensor (PIR)	30
9.1.2	Air Quality Sensor (IAQ)	32
9.1.3	Water Flow Sensor	34
9.2	Output Devices (Actuators)	34
9.2.1	Smart Lights	34
9.2.2	Fans (AC Simulation)	35
9.2.3	Solenoid Valve	35
9.3	Processing Unit	36
9.4	Communication Layer	38
9.5	Integration Workflow	38
9.6	System Diagram	39
10	Application Development	40
10.1	Application Overview and Platform	40
10.2	User Role	40
10.3	Core Features	40
10.4	User Interface Design	42
	List of Figures	43
	References	44

1. Purpose & Requirements

1.1 Purpose of the IoT System

The purpose of the Smart Mosque Management System is to design an intelligent IoT-based solution that enhances the operational efficiency of mosques by:

- Optimizing energy consumption (lighting and air conditioning)
- Conserving water usage in wudu (ablution) areas
- Monitoring and managing crowd occupancy
- Improving indoor air quality and comfort

The system leverages real-time sensor data, automated control mechanisms, and a centralized monitoring dashboard to ensure efficient resource utilization, especially during high-attendance periods such as Friday prayers and Ramadan.

1.2 Functional Requirements

These define what the system must do.

A. Energy Management

- Automatically turn ON/OFF lights and fans/AC based on:
 - Prayer times (via API)
 - Occupancy detection (motion sensors)
- Adjust energy usage dynamically depending on crowd density.

B. Occupancy Detection

- Count number of people entering/exiting using PIR sensors.
- Provide real-time hall capacity status.
- Trigger alerts if capacity exceeds safe limits.

C. Water Monitoring (Wudu Area)

- Measure water flow using flow sensors.
- Detect abnormal usage (leaks or taps left open).
- Automatically close water supply using water valves.

D. Air Quality Monitoring

- Measure air quality (e.g., CO₂, humidity, temperature).
- Trigger ventilation (fans) when thresholds are exceeded.

E. Central Dashboard

- Display:
 - Current occupancy
 - Energy consumption
 - Water usage
 - Air quality readings
- Allow manual override control for administrators.

F. Notifications & Alerts

- Send alerts for overcrowding, water leakage, poor air quality, and system faults.

1.3 Non-Functional Requirements

These define how well the system should perform.

A. Performance

- Real-time response (low latency in sensor-actuator loop)
- Efficient handling of multiple sensors simultaneously

B. Scalability

- System should support small to large mosques
- Easy addition of new sensors/devices

C. Reliability

- Continuous operation with minimal downtime
- Fault-tolerant communication between devices

D. Security

- Encrypted communication using WPA2-PSK for wireless sensor-to-gateway connections
- Dashboard access control with username/password authentication
- MAC filtering to allow only authorized IoT nodes

E. Usability

- Simple and intuitive dashboard interface
- Minimal training required for mosque staff

F. Maintainability

- Easy software/hardware updates
- Modular design for replacing faulty components

G. Cost Efficiency

- Use affordable IoT components suitable for local deployment in Egypt

1.4 Constraints and Assumptions

Constraints

- Limited budget; use cost-effective sensors
- Implementation in Cisco Packet Tracer simulation
- Dependence on internet for prayer-time API

Assumptions

- Stable local network exists in mosque
- Power supply is reliable
- Admin has basic technical knowledge

The system aligns with Egypt's sustainability goals by reducing unnecessary consumption of water and electricity in high-usage public religious spaces.

2. Process Specification

2.1 Overview

The Smart Mosque Management System operates through coordinated processes that continuously collect sensor data, analyze conditions, and trigger automated actions. These processes ensure efficient energy usage, water conservation, and safe occupancy management. The system follows a Sense → Process → Act → Monitor pipeline.

2.2 Core System Processes

2.2.1 Occupancy Detection Process

Objective: Monitor the number of people inside the mosque.

Workflow:

1. PIR sensors at entrances detect entry/exit events.
2. System increments/decrements occupancy count.
3. Data is sent to the central controller.

4. If occupancy exceeds threshold, trigger alert on dashboard.
5. Occupancy data is used by other processes (energy control).

2.2.2 Energy Management Process

Objective: Optimize electricity usage based on prayer times and occupancy.

Workflow:

1. System retrieves prayer times from API.
2. Motion sensors detect presence in prayer hall.
3. Decision logic:
 - IF (Within Prayer Window OR Motion Detected), then turn ON lights and AC/fans.
 - ELSE, turn OFF devices.
4. Optionally adjust intensity based on crowd density.

2.2.3 Wudu Water Monitoring Process

Objective: Prevent water waste in ablution areas.

Workflow:

1. Flow sensors measure water usage.
2. System checks for continuous flow without interruption.
3. If abnormal usage is detected:
 - Assume tap left open or leak.
 - Activate water valve to shut water.
4. Log water consumption data.

2.2.4 Air Quality Monitoring Process

Objective: Maintain comfortable and safe air conditions.

Workflow:

1. Sensors read CO₂, humidity, and temperature.
2. Compare readings with predefined thresholds.
3. If air quality is poor, activate ventilation.
4. Update dashboard with real-time values.

2.2.5 Central Monitoring and Control Process

Objective: Provide full system visibility and control.

Workflow:

1. Collect data from all subsystems.
2. Display occupancy, energy usage, water consumption, and air quality on web dashboard.
3. Allow admin override and manual ON/OFF operations.
4. Optionally store historical data.

2.3 Integrated System Flowchart

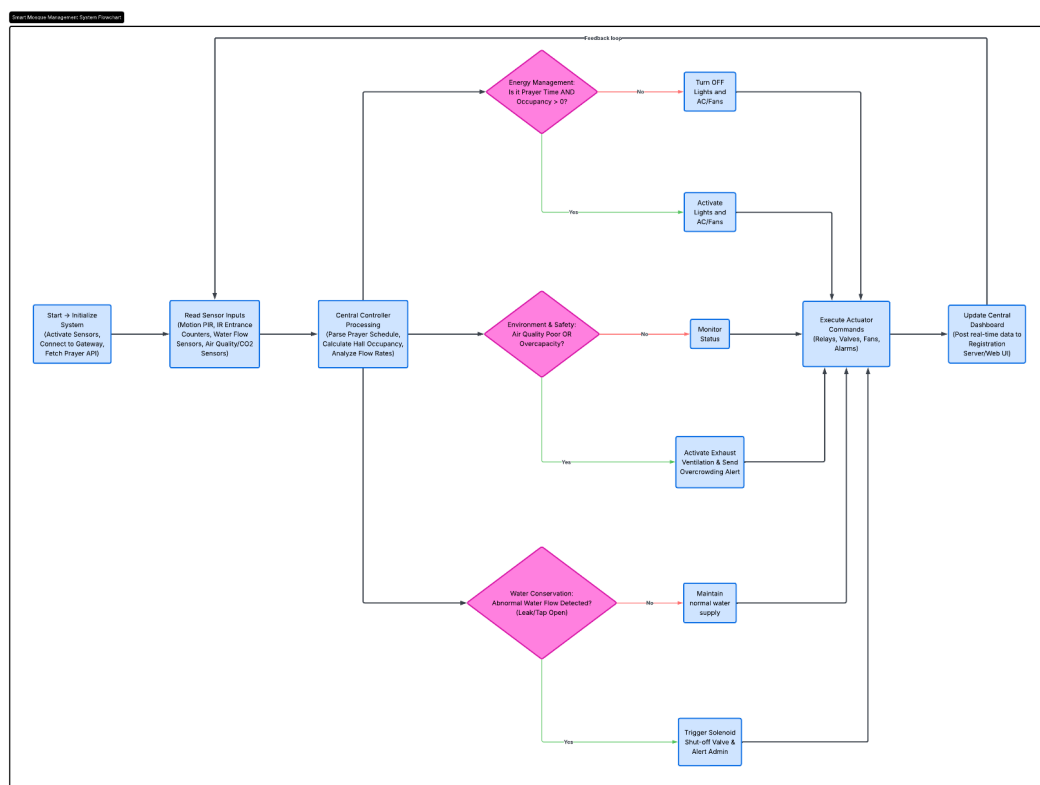


Figure 1: Integrated system flowchart

2.4 Process Interaction Notes

- Occupancy data affects energy decisions.
- Air quality can override energy logic (force ventilation ON).
- Water system is independent but logged centrally.
- All processes communicate via a central controller (MCU/SBC).

The system processes are designed as loosely coupled modules to ensure scalability, maintainability, and real-time responsiveness.

3. Domain Model Specification

3.1 Entities

Core System Entities:

- Mosque
- Zone (Prayer Hall / Wudu Area)

Sensors:

- Motion Sensor
- IR Counter Sensor
- Air Quality Sensor
- Water Flow Sensor

Actuators:

- Light
- Fan / AC
- Water Valve

System Components:

- Controller (MCU/SBC)
- IoT Server
- Dashboard (Web App)
- Admin

3.2 Attributes for each Entity

Mosque

- mosqueID
- name
- location

Zone

- zoneID
- type (Prayer Hall / Wudu Area)
- capacity

Sensor

- sensorID
- type (motion, PIR, air, water)
- status (active/inactive)
- value (reading)

Actuator

- actuatorID
- type (light, fan, valve)
- status (ON/OFF)

Controller

- controllerID
- processingLogic
- status

IoT Server

- serverID
- database
- connectionStatus

Dashboard

- dashboardID
- displayData
- controlOptions

Admin

- adminID
- username
- password

3.3 Relationships

- Mosque contains Zones
- Zone contains Sensors
- Zone contains Actuators
- Sensors send data to Controller
- Controller controls Actuators
- Controller communicates with IoT Server
- IoT Server updates Dashboard
- Admin interacts with Dashboard
- Dashboard sends commands to Controller

3.4 UML Diagram

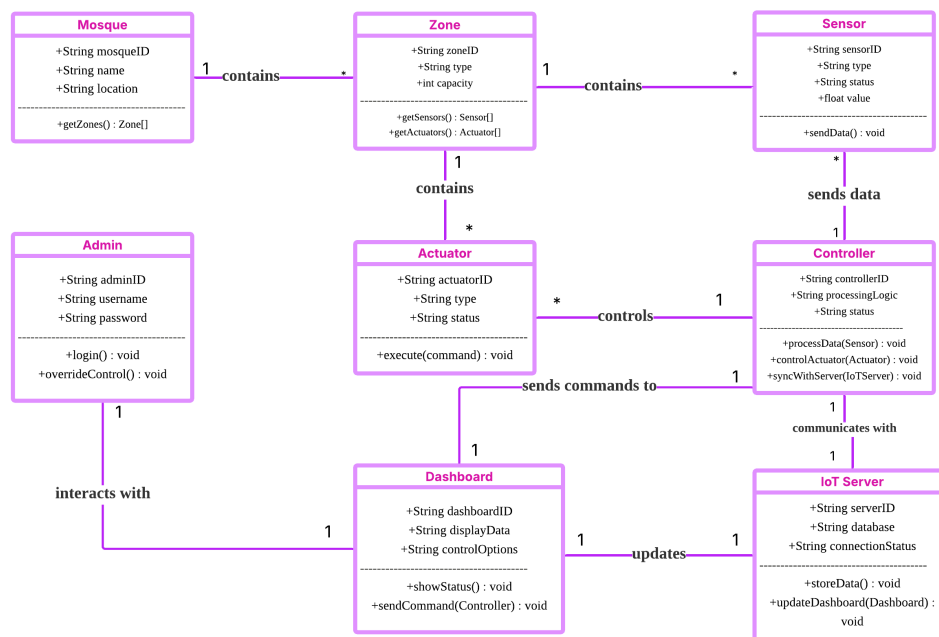


Figure 2: UML domain model diagram

4. Information Model Specification

4.1 Types of Data in System

4.1.1 Sensor Data (Raw Data)

Occupancy Data

- peopleCount (int)
- entry/exit events

Motion Data

- motionDetected (boolean)

Air Quality Data

- CO₂Level (ppm)
- temperature (°C)
- humidity (%)

Water Data

- flowRate (L/min)
- usageDuration (seconds)

4.1.2 Processed Data

- occupancyStatus (normal / full / overcrowded)
- airQualityStatus (good / moderate / poor)
- waterUsageStatus (normal / leak detected)
- energyState (ON / OFF)

4.1.3 System Data (Stored)

- historical water usage
- energy consumption logs
- occupancy logs
- alerts history

4.1.4 Control Data (from Admin)

- manual override commands
- turn ON/OFF devices

4.2 Data Flow

4.2.1 Main Flow

Sensors → Controller → IoT Server → Dashboard

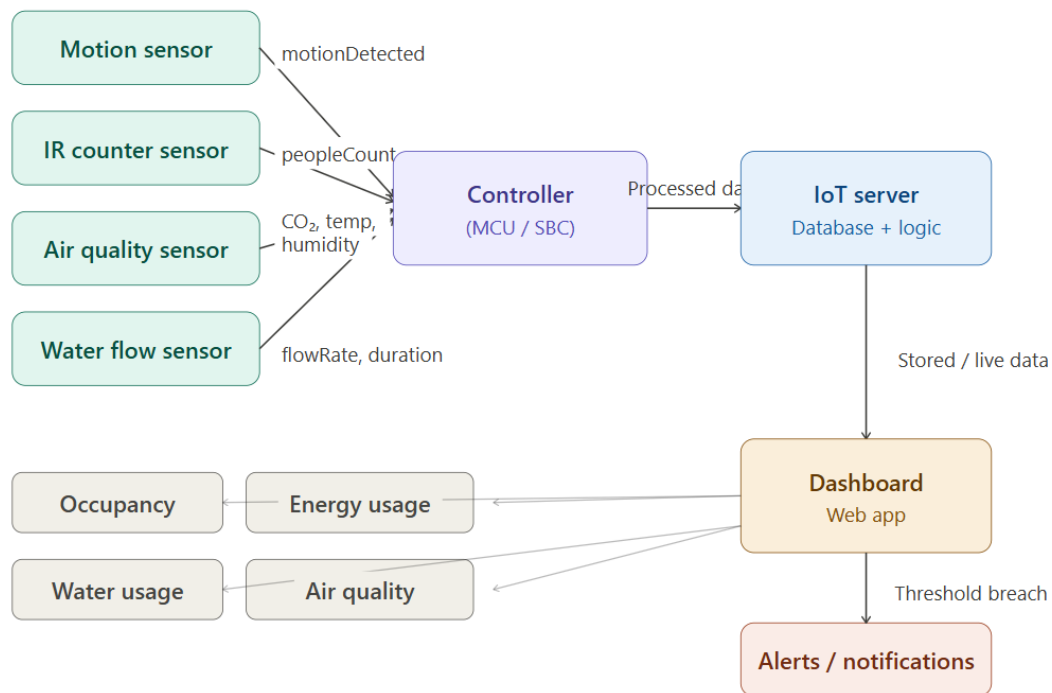


Figure 3: Main data flow

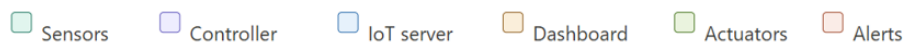


Figure 4: Main flow legend

4.2.2 Control Flow

Admin → Dashboard → Controller → Actuators

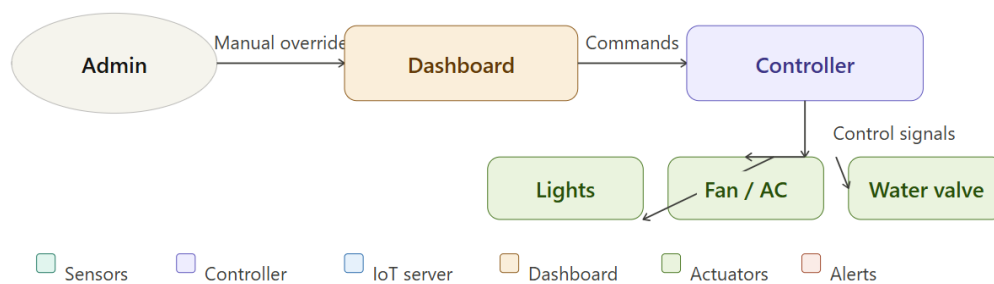


Figure 5: Control flow

5. Service Specifications

5.1 Overview

The Smart Mosque Management System exposes seven core services. Each service is a defined unit of functionality with clear inputs, outputs, triggers, and interfaces. Services are implemented on the ESP32 controller and exposed to the dashboard via the local network.

5.2 Service List

Service ID	Service Name	Type	Trigger
SVC-01	Prayer Time Sync	Data Acquisition	Scheduled – once daily
SVC-02	Occupancy Tracking	Sensing	Continuous (event-driven)
SVC-03	Energy Control	Actuation	Event-driven
SVC-04	Water Monitoring & Control	Sensing + Actuation	Continuous
SVC-05	Air Quality Monitoring	Sensing + Actuation	Periodic (every 60s)
SVC-06	Alert & Notification	Notification	Threshold-triggered
SVC-07	Dashboard Monitoring & Override	Application + Control	On-demand

Table 1: Core service list

5.3 Service Descriptions

SVC-01 – Prayer Time Sync Service

- Purpose: Retrieves accurate daily prayer times to anchor energy automation logic. Without this, the system cannot distinguish active prayer windows from idle periods.
- Interface: HTTP GET request to a prayer time API (e.g., aladhan.com/v1/timingsByCity?city=Cairo&count=1) once per day at midnight.
- Output: Five daily timestamps (Fajr, Dhuhr, Asr, Maghrib, Isha) stored in ESP32 local memory.
- Fallback: If the API is unreachable, the system uses previous day's cached schedule. The drift between consecutive days for Egyptian cities is under 2 minutes – acceptable for this use case.

Data produced:

```
{
  "fajr": "04:12",
  "dhuhr": "12:01",
  "asr": "15:28",
  "maghrib": "18:44",
  "isha": "20:10"
}
```

```
}

```

SVC-02 – Occupancy Tracking Service

- Purpose: Counts the number of people inside the prayer hall in real time to support energy decisions and crowd management.
- Interface: PIR sensor events at entry and exit points → ESP32 counter logic → count exposed via local API endpoint polled by dashboard every 30 seconds.
- Operations:
 - `increment_count()` – triggered on entry detection
 - `decrement_count()` – triggered on exit detection; floor is 0 (never negative)
 - `get_current_count()` – returns current integer count and status flag
- Output fields:

Field	Type	Description
occupancy_count	int	Current number of people in hall
capacity_percent	float	$\text{Count} \div \text{max capacity} \times 100$
status	enum	NORMAL / HIGH (>80%) / FULL (=max)

Table 2: Occupancy service output fields

Alert integration: When status transitions to HIGH or FULL, SVC-06 is triggered automatically.

SVC-03 – Energy Control Service

- Purpose: Automatically activates and deactivates lights and fans/AC per zone based on prayer window status and occupancy presence.
- Interface: ESP32 reads state from SVC-01 and SVC-02, applies decision logic, and sends GPIO signals to relay modules controlling lights and fans.
- Decision logic:
 - IF (`current_time` is within `prayer_window`) AND (`occupancy` > 0): `activate(lights, fans)` for active zone
 - ELSE: `deactivate(lights, fans)` for active zone
- IF (`override_active`): ignore above → apply override command from SVC-07
- Operations:

Note: Air quality readings from SVC-05 can override the fan state independently – poor air quality activates fans regardless of prayer window status.

Operation	Description
activate_zone(zone_id, device_type)	Turns on lights or fans in a specific zone
deactivate_zone(zone_id, device_type)	Turns off lights or fans in a specific zone
get_device_status(device_id)	Returns current ON/OFF state of a device

Table 3: Energy control operations

SVC-04 – Water Monitoring and Control Service

- Purpose: Monitors water flow in wudu area, detects leaks or taps left open, and automatically closes solenoid valve after defined inactivity timeout.
- Interface: Water flow sensor (pulse output) → ESP32 pulse counter → solenoid valve GPIO control.
- Key operations:

Operation	Description
monitor_flow(tap_id)	Continuously reads flow rate in L/min
detect_anomaly()	Flags non-zero flow after 30s of no motion nearby
close_valve(tap_id)	Sends close signal to solenoid valve
open_valve(tap_id)	Reopens valve (manual or after reset)
get_water_usage()	Returns cumulative usage in liters per session/day

Table 4: Water monitoring operations

Key thresholds:

Condition	Threshold	Action
Normal use	1–3 L/min	No action
Inactivity timeout	Flow > 0 with no motion for > 30s	Close valve + alert
Leak flag	Flow above normal with no motion	Close valve + high-priority alert

Table 5: Water monitoring thresholds

Failsafe: If flow sensor goes offline, valve defaults to closed and admin is notified.

SVC-05 – Air Quality Monitoring Service

- Purpose: Monitor indoor CO₂ levels, temperature, and humidity. Activates ventilation fans when readings exceed safe thresholds, independent of prayer time logic.
- Interface: IAQ sensor (I²C/UART) → ESP32 → fan relay GPIO + dashboard update.

- Polling interval: Every 60 seconds. Suitable for CO₂ and climate parameters, which change gradually rather than instantaneously.

Output pushed to dashboard every 60s:

```
{
  "co2_ppm": 870,
  "temperature_c": 27,
  "humidity_percent": 58,
  "air_quality_status": "MODERATE"
}
```

Status scale: GOOD (0–50 AQI) / MODERATE (51–100) / POOR (101–150) / UNHEALTHY (151+)

Parameter	Normal	Action Threshold	Action
CO ₂	< 1000 ppm	> 1000 ppm	Activate fans
Temperature	18–26 °C	> 28 °C	Activate fans
Humidity	30–60%	> 65%	Activate fans

Table 6: Air quality thresholds

SVC-06 – Alert and Notification Service

- Purpose: Detects threshold breaches across all subsystems and pushes real-time alerts to admin dashboard.
- Interface: Internal event bus – ESP32 publishes events; dashboard polls the alert queue.
- Deduplication: Same alert type for same device/zone fires at most once per 5 minutes.

Alert definitions:

Alert ID	Trigger Condition	Severity
ALT-01	Occupancy > 80% of max capacity	WARNING
ALT-02	Occupancy = max capacity	CRITICAL
ALT-03	Water flow with no motion > 30s	WARNING
ALT-04	Leak pattern detected (high flow, no motion)	CRITICAL
ALT-05	Air quality status = POOR or UNHEALTHY	WARNING
ALT-06	Sensor offline / no response	FAULT
ALT-07	Actuator failed to respond after 3 retries	FAULT

Table 7: Alert definitions

Alert record stored:

```
{
  "alert_id": "ALT-03",
  "timestamp": "2026-04-25T13:42:00",
  "zone": "wudu_area",
  "status": "ACTIVE"
}
```

SVC-07 – Dashboard Monitoring and Manual Override Service

- Purpose: Provides the mosque administrator with a unified view of all system state and the ability to manually override any automated decision.
- Interface: Web dashboard (hosted on local server/CPT Registration Server) polling ESP32 REST endpoints every 30 seconds. Override commands sent via HTTP POST.

Data displayed:

Panel	Data shown
Occupancy	Live count, capacity bar, status flag
Energy	Device ON/OFF state per zone, prayer schedule
Water	Flow rate per tap, daily usage total, valve state
Air Quality	CO ₂ , temperature, humidity, AQI status
Alerts	Active alert queue with timestamps

Table 8: Dashboard data panels

Override operations:

Operation	Description
override_on(device_id, duration_min)	Forces device ON for N minutes
override_off(device_id)	Forces device OFF regardless of logic
clear_override(device_id)	Returns device to automatic mode

Table 9: Override operations

Safety constraint: Override automatically expires after the set duration. Indefinite override requires admin re-confirmation every 60 minutes.

Use case: Ramadan Tarawih prayers, Eid prayers, and other non-standard events where standard prayer-time logic is insufficient.

6. IoT Level Specification

The Smart Mosque Management System is structured across five IoT levels, following a standard layered architecture. Each layer has a defined responsibility and communicates with adjacent layers.

6.1 Level 1 – Perception Layer

Role: Direct interaction with the physical environment.

Sensors

Device	Placement	Measurement
PIR Motion Sensor	Main entrance, exit, hall zone	Entry/exit events, zone presence
Water Flow Sensor	Wudu tap inlet pipe	Flow rate (L/min), pulse output
PIR Motion Sensor (Wudu)	Ablution area	Presence near taps
IAQ Sensor	Prayer hall near ventilation	CO ₂ , temperature, humidity

Table 10: Perception layer sensors

Actuators

Device	Placement	Controlled Action
Smart Relay (Lights)	Per hall zone	ON/OFF lighting
Smart Relay (Fans/AC)	Per hall zone, wudu area	ON/OFF ventilation
Solenoid Valve	Wudu tap supply line	Open/close water flow

Table 11: Perception layer actuators

6.2 Level 2 – Network Layer

Role: Local connectivity and data transport.

- Hardware: Local Wi-Fi access point for ESP32 nodes and local server
- Protocol: IEEE 802.11 Wi-Fi with WPA2-PSK
- Topology: Star
- Segmentation: IoT network isolated from guest/user devices

Key decision: Core control loops are local and do not depend on internet connectivity.

6.3 Level 3 – Edge Processing Layer

Role: Real-time control and decision-making.

Hardware: ESP32 microcontrollers per functional zone.

Function	Description
Sensor polling	Reads PIR, IAQ, and flow sensor inputs at configured intervals
Occupancy counting	Increments/decrements people count based on entry/exit events
Prayer window evaluation	Compares current time against cached prayer schedule
Energy control logic	Combines occupancy and prayer window state for relays
Water control logic	Detects anomalies and closes valve when required
Air quality control	Polls IAQ every 60s and activates fans on threshold breach
Alert generation	Publishes threshold breach events to data layer

Table 12: Edge processing responsibilities

Failsafe: If server connectivity is lost, edge controllers continue autonomous operation.

6.4 Level 4 – Data Accumulation Layer

Role: Aggregate processed data, store logs, and serve dashboard queries.

- CPT simulation backend: Registration Server
- Real deployment option: local server (e.g., lightweight REST + database)
- Retention: rolling 30-day window with optional archival

6.5 Level 5 – Application Layer

Role: Human-facing dashboard for monitoring and control.

- Web dashboard on local network
- Single admin role
- Functions: live monitoring, alert display, schedule view, manual override

6.6 Level Interaction Summary

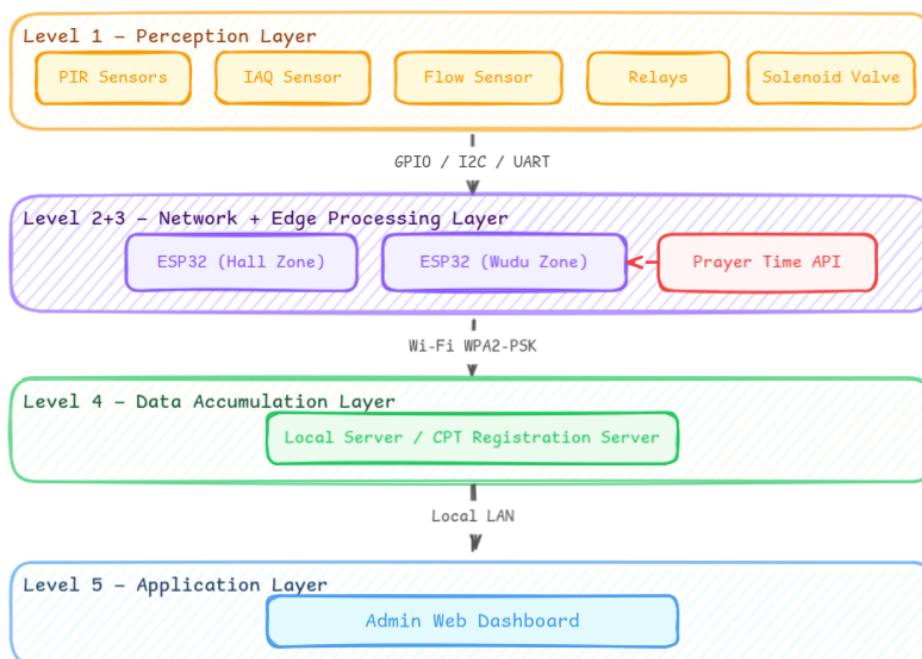


Figure 6: IoT system layer interactions

7. Functional View Specification

7.1 Overview

The functional view breaks the system into independent blocks by behavior. Each block has defined inputs, outputs, and responsibilities, with clear data flows for modularity and maintainability.

7.2 Functional Block Descriptions

Block 1 – Sensing Block

Collects raw data from the mosque environment.

Sub-function	Sensor	Interval	Output
Entry/exit detection	PIR (entrance)	Event-based	Increment/decrement event
Zone presence detection	PIR (hall zones)	Every 5s	Presence flag per zone
Water flow measurement	Flow sensor	Every 1s	Flow rate (L/min)
Tap-area presence	PIR (wudu)	Event-based	Presence flag
Air quality reading	IAQ sensor	Every 60s	CO ₂ , temperature, humidity

Table 13: Sensing block functions

Failure behavior: Offline sensors are flagged to processing, then safe defaults are applied.

Block 2 – Processing Block

Decision core running on ESP32.

- Occupancy counter with floor at zero.
- Prayer window evaluator from cached schedule.
- Energy decision engine for lights/fans.
- Air quality decision engine independent from prayer window.
- Water anomaly detection and valve control.
- State manager as single source of truth.

Block 3 – Actuation Block

Executes commands on relays and valve, with confirmation and retries.

Device	Command Type	Confirmation
Smart relay (lights)	ON/OFF per zone	GPIO acknowledgment
Smart relay (fans/AC)	ON/OFF per zone	GPIO acknowledgment
Solenoid valve	OPEN/CLOSE per tap	GPIO acknowledgment

Table 14: Actuation block control and feedback

Block 4 – Data and Storage Block

Persists time-series data for dashboard and analytics.

Data Type	Logging Frequency	Retention
Occupancy count	Every 30s	30-day rolling
Water usage per tap	Per session + hourly total	30-day rolling
Device state changes	On every transition	30-day rolling
Air quality readings	Every 60s	30-day rolling
Alert records	On every alert event	30-day rolling

Table 15: Stored operational data

Block 5 – Alert Block

Evaluates alert conditions, deduplicates repeated alerts, and pushes active alerts to dashboard.

Block 6 – Dashboard Block

Provides monitoring panels and manual override controls for admin users.

Block 7 – External API Block

Fetches daily prayer times and caches them for resilience.

7.3 Inter-Block Data Flow Summary

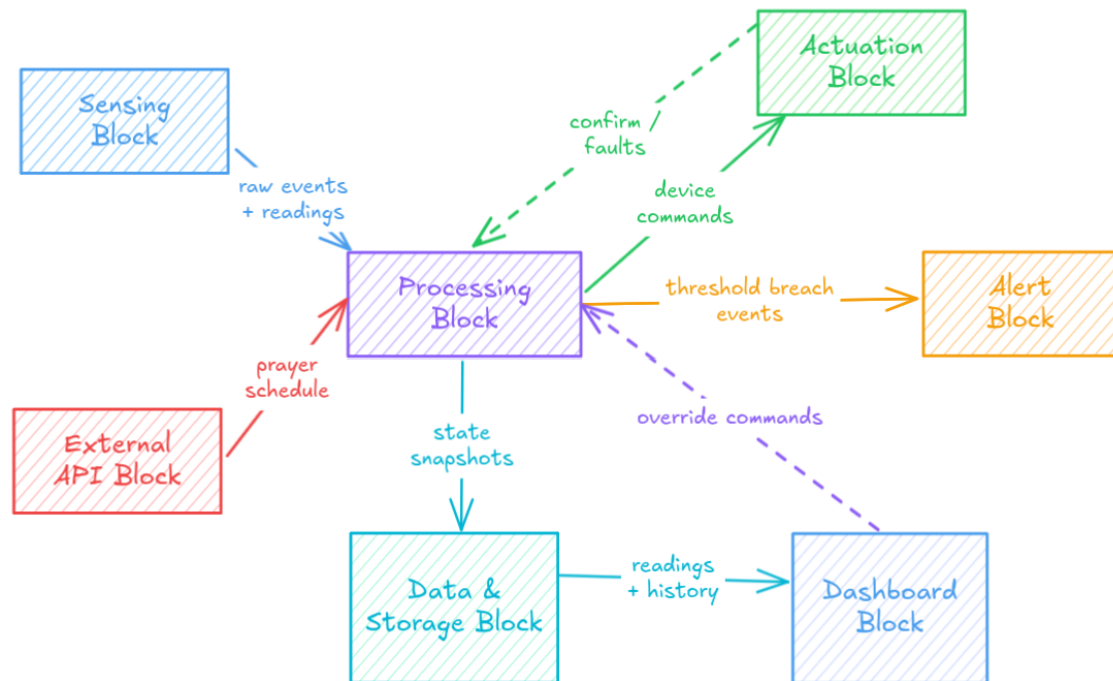


Figure 7: Inter-block data flow

8. Operational View Specification

8.1 System Performance

8.1.1 Response Time

The system operates in near real-time. Motion events trigger lighting and ventilation in approximately 1–2 seconds. In the wudu area, abnormal continuous flow triggers valve closure in approximately 2–3 seconds to reduce waste while limiting false positives.

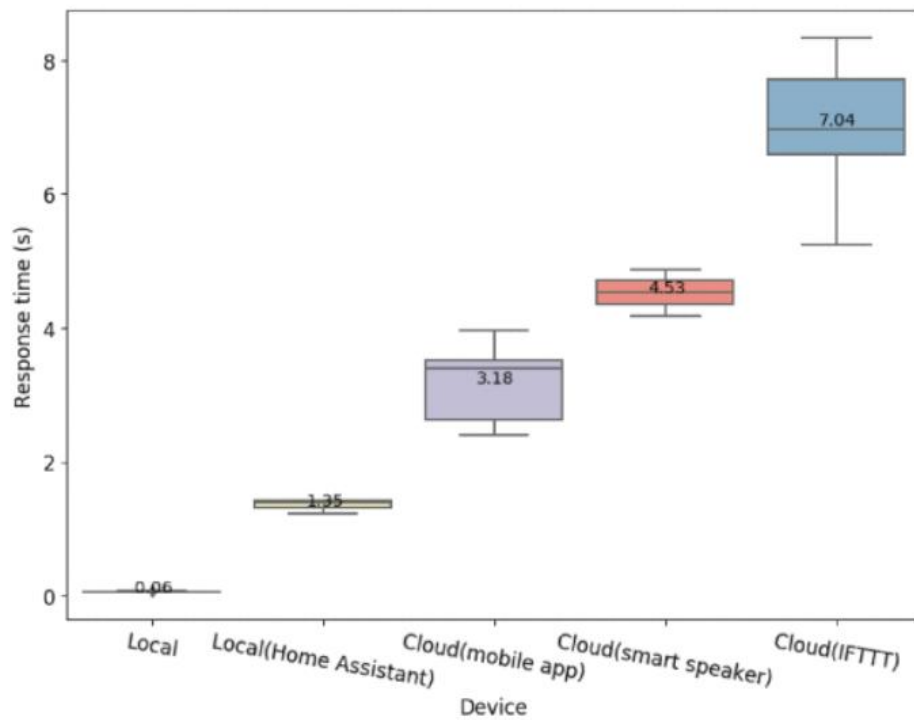


Figure 8: Operational response-time trend

Air quality is monitored periodically and ventilation is adjusted when thresholds are exceeded. Temperature and humidity are also tracked to maintain comfort. The system handles simultaneous sensor inputs during peak times without noticeable degradation.

8.1.2 Data Update Frequency

- Occupancy: real-time update per entry/exit event.
- Air quality: every 1 minute (balanced responsiveness and stability).
- Water usage: continuously monitored in real time.

8.1.3 Throughput

The system processes motion, water-flow, and air-quality streams in parallel, enabling simultaneous monitoring and decision-making during high-demand periods such as Friday prayer and Ramadan.

The system is designed to handle multiple sensors at once by collecting motion, water-flow, and air-quality data in parallel and sending each reading to the controller without blocking the others. This means the mosque can monitor occupancy, water usage, and ventilation conditions simultaneously, which is important when several events happen at the same time. It can also handle continuous data

flow during peak time, such as Friday prayer or Ramadan, because the sensors keep updating while the system processes live changes in crowd density and resource usage. As a result, the dashboard remains responsive even under heavy activity, and the mosque administration can make quick decisions without losing real-time visibility.

8.1.4 Efficiency

Lights and fans operate only when needed based on occupancy and schedule, and water control logic closes supply during leak-like behavior to reduce waste.

The system is efficient because it avoids unnecessary activation of devices. Lights and fans turn on only when occupancy is detected and the prayer-time window requires them, which reduces energy waste and keeps power use aligned with actual demand. The system also improves water efficiency by monitoring flow in real time and shutting off the supply when it detects leaks or taps left running, which helps prevent unnecessary water waste.

8.2 System Reliability

8.2.1 Sensor Failure Handling

Fallback mechanisms keep operation safe during sensor faults.

- Motion sensor failure: admin manual override controls lighting and fans.
- Flow sensor failure: manual valve override + admin alert.
- Air-quality sensor failure: ventilation follows safe default schedule.

8.2.2 Network Failure

Core control continues locally if network/server connectivity is lost. Dashboard synchronization resumes automatically after restoration.

8.2.3 Power Failure

On recovery, the system returns to predefined safe startup states. Critical monitoring may be maintained through optional backup power.

8.3 System Scalability

8.3.1 More Sensors

Additional occupancy, flow, and IAQ sensors can be integrated with the same controller logic.

8.3.2 Spreading to Other Mosques

The architecture can be replicated across multiple mosques, each with local operation and data separation.

8.3.3 Modular Architecture

Occupancy, water, air-quality, and dashboard modules can evolve independently, reducing maintenance risk and easing upgrades.

8.4 System Security

8.4.1 Network Protection

- Segmented network for IoT devices
- Restricted dashboard/control access
- Firewall/gateway policy enforcement
- Monitoring for suspicious behavior and repeated login attempts

All IoT devices communicate through a secure local network rather than being openly exposed. This reduces the risk of external interference and makes it easier to control access to the mosque's sensors and actuators. The system ensures:

- Sensors, controllers, and actuators are on a separate, secured local network segment from general user devices.
- Restricted remote access so only approved users (admins) or services can reach the dashboard and device controls.
- Firewall and gateways rules are implemented to reduce the chance of unauthorized access.
- Network traffic is constantly monitored for unusual device behavior or repeated login attempts.

8.4.2 Access Control

Role-based permissions, authenticated overrides, and action logging are applied for accountability.

Only authorized users can view the dashboard or override systems such as lights, fans, and water valves. Role-based access control is a standard way to assign permissions, so the mosque administrator has full control while other users have limited or read-only access. This is implemented by giving the mosque administrator full control, while staff or guests get limited or read-only access. Manual overrides are only done through login credentials so devices cannot be changed by unauthenticated users. The system also uses separate permission levels for monitoring, control, and maintenance actions. For more protection, the system logs every override action so the system keeps a record of who changed what and when.

Authentication flow:

User → Login Page → Authentication Check → Access Granted → Dashboard

8.4.3 Data Encryption

WPA2-PSK secures Wi-Fi communication between sensors, controllers, and dashboard. AES encryption protects traffic confidentiality and integrity.

Data transmitted within the system is protected using WPA2-PSK (Wi-Fi Protected Access 2 - Pre-Shared Key) encryption over the wireless network. WPA2-PSK is a Wi-Fi security method that protects a network with a shared password and encrypts the data sent over it and does not require complex key management while still maintaining a strong level of confidentiality and integrity for transmitted data. This ensures that communication between sensors, the controller, and the dashboard is secured against unauthorized access.

It works by using a shared password to authenticate devices and AES encryption to protect the Wi-Fi traffic, with the password acting as the secret key that both devices and the access point use to connect securely. AES encryption is a symmetric encryption method that scrambles data into unreadable ciphertext using the same secret key for both encryption and decryption. By encrypting data at the network level, the system helps prevent attackers from intercepting or modifying sensitive information such as occupancy counts, water usage data, and device control commands.

What the system does:

- Secures wireless communication between sensors (motion, water flow, air quality) and the controller using WPA2-PSK.
- Protects local dashboard traffic transmitted over the Wi-Fi network.
- Allows only authorized devices with the correct network credentials to join the network.
- Provides a balance between security and performance, making it suitable for embedded IoT devices.

8.4.4 Device Filtering

MAC allow-list filtering blocks unknown devices from joining. It is used together with WPA2-PSK and access control (not standalone security).

Device filtering adds a basic access-control layer by allowing only approved IoT nodes to join the mosque network. In this project, MAC address filtering is used to create an allow list of trusted sensors, controllers, and dashboard devices, so any unregistered device is blocked from connecting. This is useful in a public environment because it reduces the chance of unknown devices joining the

system and interfering with occupancy, water, or ventilation control.

- Allow list only trusted devices: The router or access point can be configured so only the MAC addresses of authorized IoT nodes are permitted to connect.
- Block unknown devices: Any device not on the list is denied network access, which helps keep the system limited to approved controllers and sensors.
- Useful for IoT deployment: MAC filtering is practical in small, controlled networks like a mosque IoT setup because the number of devices is known and relatively stable.
- Not a standalone security control: MAC addresses can be spoofed, so this method should be used together with WPA2-PSK and role-based access control rather than as the only protection.
- Supports safer maintenance: When a new sensor or controller is added, its MAC address can be registered first, which gives the administrator a simple way to manage device enrollment.

8.4.5 Privacy Protection

The system stores aggregate occupancy counts only and avoids personal identification. PIR-based sensing does not capture images.

8.5 Energy Efficiency

Prayer-time-aware control avoids continuous operation outside worship periods.

8.5.1 Prayer Time Control

Prayer-time automation helps the system avoid continuous operation outside scheduled worship periods. By linking the control logic to prayer times using APIs, the mosque can keep lighting and ventilation off during low-use periods, then activate them only when worshippers begin to gather. This reduces wasted power while keeping the environment comfortable and ready for prayer.

8.5.2 Occupancy-Based Activation

Devices activate only when people are present, reducing unnecessary consumption.

Occupancy detection adds a second layer of control, so the system does not turn on devices unless people are present. If the prayer hall is empty, the lights and fans remain off even if the prayer window is active, which prevents unnecessary consumption. This is important because a mosque may have different attendance levels depending on the prayer, day, or season.

8.5.3 Automatic Shutdown

Lights and fans turn off after inactivity timeout, minimizing waste.

The system also improves efficiency by shutting down devices automatically after inactivity. If no motion is detected for a set period, the lights and fans can turn off without manual intervention, which helps prevent energy waste when people leave early or when only part of the hall is being used. This makes the system both practical and energy-conscious, especially for high-traffic periods.

8.6 Safety Considerations

8.6.1 Device Overheating

Thermal-safe operation requires ventilation, safe power design, and alerts/shutdown under abnormal heating.

The system includes safeguards to prevent overheating of devices such as controllers, relays, and power modules. Since these components may run for long periods during peak prayer times, the design should allow for proper ventilation, safe power handling, and automatic shutdown or alerting if temperature rises beyond safe limits. This helps protect the hardware and improves long-term reliability.

8.6.2 Water Valve Reliability

Valve closure is validated over a short window to avoid false shutdown during normal use interruptions.

The water valve is controlled carefully so it does not shut off incorrectly while someone is still using water. To avoid this, the system should confirm abnormal flow for a short validation period before closing the valve, rather than reacting to a brief or normal interruption in use. This reduces false shutdowns and ensures that the area remains safe and practical for worshippers.

9. Device and Component Integration

9.1 Input Devices (Sensors)

9.1.1 Motion Sensor (PIR)

A Passive Infrared (PIR) sensor detects changes in infrared radiation emitted by warm bodies. It is compact, low-cost, and energy-efficient.



Figure 9: PIR motion sensor (left) and PIR sensing grid illustration (right)

Sensor details:

- Detection range: up to 7 meters
- Detection angle: 110 degrees
- Operating voltage: DC 4.5V–12V
- Output signal: 3.3V digital output
- Delay time: 0.3 seconds to 5 minutes (adjustable)
- Operating temperature: -15°C to $+70^{\circ}\text{C}$
- Sensitivity: adjustable

Working principle summary:

1. Infrared detection: The PIR sensing element continuously detects infrared radiation from the environment. When a warm object crosses the field of view, the detected radiation level changes from one zone to another, producing a measurable differential signal. This transition is the key trigger used by the sensor to recognize motion.

2. Sensor structure: Uses dual sensing zones and lens focusing.

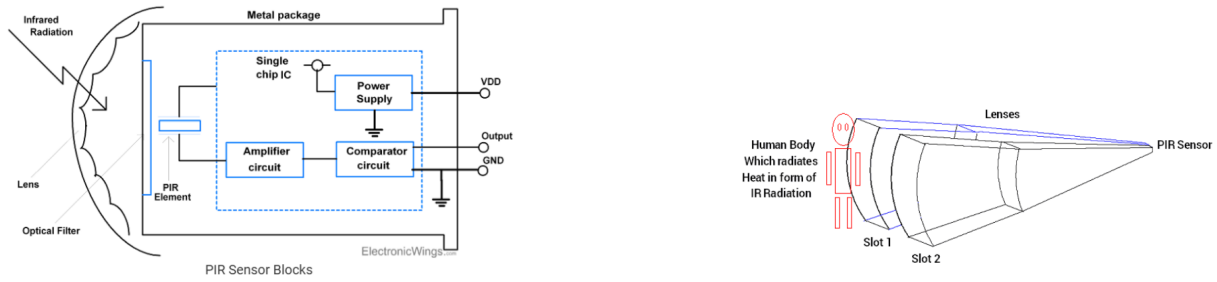


Figure 10: Infrared detection concept (left) and PIR sensor structure (right)

3. Idle state: Balanced sensing zones keep the output stable.

4. Motion detection: As a person moves across the sensing area, one PIR segment receives higher infrared energy than the other. The sensor circuit amplifies this difference and interprets it as movement.

5. Output generation: After threshold comparison, the sensor produces a digital output pulse. This output is forwarded to the controller to trigger occupancy and automation logic in real time.

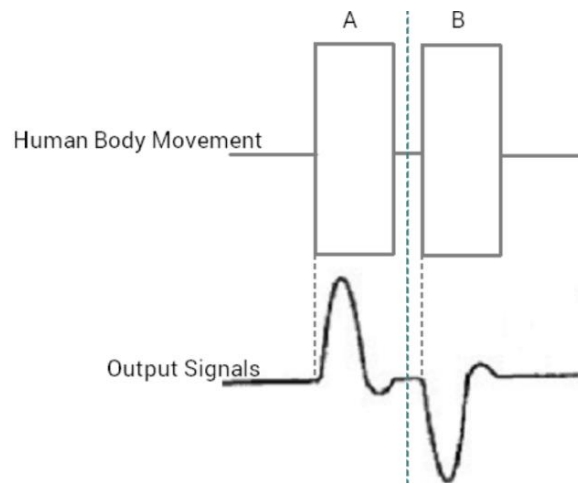


Figure 11: Motion detection behavior

PIR should be installed away from Wi-Fi antennas to reduce electromagnetic interference risk.

9.1.2 Air Quality Sensor (IAQ)



Figure 12: Indoor air quality sensor module

IAQ sensing supports ventilation control by monitoring gases and climate parameters.

Key IAQ factors:

- Chemical pollutants (CO₂, VOCs, NO₂)
- Biological contaminants (mold, bacteria, allergens)
- Particulate matter (PM2.5, PM10)
- Environmental conditions (humidity, temperature, pressure)

Common pollutants and impact:

Pollutant	Description	Health Effects
CO ₂	Ventilation indicator	Fatigue, headache, lower productivity
VOCs	Emitted by paints/cleaners/plastics	Irritation, dizziness, long-term risk
PM2.5/PM10	Fine dust and smoke particles	Respiratory and cardiovascular burden
CO	Incomplete combustion product	Severe toxicity at high concentration
Formaldehyde	Emitted by furniture/insulation	Carcinogenic, irritation
NO ₂ /O ₃	Combustion and outdoor sources	Respiratory distress
Humidity	Moisture level	Mold growth, discomfort
Temperature	Thermal condition	Fatigue, reduced comfort

Table 16: IAQ factors and effects

IAQ sensors require periodic calibration (typically every 6–12 months).

A good quality sensor offers accurate reading, but like any sensor, IAQ comes with some challenges like:

- Sensor drift and degradation over time
- Cross-sensitivity (e.g., VOC sensors reacting to humidity)
- Need for periodic calibration
- Data overload in multi-sensor networks
- False readings due to temperature/humidity interference

It is important to calibrate it every 6-12 months to ensure good quality reading.

Working principles of IAQ sensors:

- Sensing: Each sensor type uses a specific physical or chemical method to detect its target pollutant.
- Signal Processing: Detected signals are converted into electrical output (analog or digital).
- Calibration: Sensors are calibrated for accuracy, often using reference gases.
- Data Output: The processed data is sent via I2C, UART, Modbus, or wireless protocols (Wi-Fi, Zigbee, LoRa).
- Display and Analysis: Results are visualized on display or uploaded to the cloud for remote monitoring.

Commonly, the output of an IAQ sensor can range from 0-50 (good), 51-100 (moderate), 101-150 (unhealthy for sensitive groups), and 151-200 (unhealthy).

9.1.3 Water Flow Sensor



Figure 13: Water flow sensor

Water flow sensors measure instantaneous flow rate and cumulative usage, producing pulse output suitable for MCU integration. This allows the controller to detect abnormal continuous flow and apply water-saving actions when needed.

The sensor is usually installed at the water inlet or at the opening of the pipe. The required supply voltage is usually 5V to 18V DC. Water flow sensors can be used with hot, cold, warm, clean, or dirty water, depending on the sensor type. In our case, it should be enough for clean water of all temperatures. These sensors can be easily connected to microcontrollers such as Arduino.

Working principle summary:

- Water drives an internal rotor.
- Hall-effect sensing converts rotor speed to pulse signal.
- Pulse frequency maps to flow rate.
- Controller computes usage and triggers control logic.

9.2 Output Devices (Actuators)

9.2.1 Smart Lights

Smart lights are used as an output device to provide illumination in the mosque when needed. Their main role is to support comfort and visibility during prayer times while reducing unnecessary energy use during periods of low occupancy.

Working principle:

- The system checks whether the prayer time is active.
- It also checks whether a PIR sensor has detected people in a specific area of the mosque.
- If both conditions are true, the lights turn on automatically.
- If no people are detected, the lights remain off even during the prayer window.
- After the prayer time ends, the lights turn off automatically (if no people detected).
- This ensures that lighting is provided only when the prayer hall is in use.

How it functions in the project: When the PIR sensor detects movement or presence in a section of the mosque, the system interprets this as occupancy in that area. If this happens during the prayer period, the smart lights are activated to provide adequate lighting for worshippers. If the area becomes empty, the lights are switched off after a short delay to avoid unnecessary power consumption. This makes the lighting system both responsive and energy efficient, since it operates only when people are present and prayer time requires it.

9.2.2 Fans (AC Simulation)

Fans and AC units act as output devices that maintain a comfortable environment in the mosque by responding to indoor air quality readings such as temperature, humidity, CO₂, and other pollution indicators. Their role is to improve ventilation when the prayer hall becomes crowded or the air becomes warm or stale, and to reduce operation when conditions return to normal.

Working principle:

- The IAQ sensor continuously measures temperature, humidity, and CO₂ levels inside the mosque.
- If the readings show that the air is getting warm, humid, or poorly ventilated, the controller sends a signal to activate the fans or AC.
- If the air quality stays within acceptable limits, the fans and AC remain off or operate at a lower level.
- This allows the ventilation system to respond automatically to real environmental conditions rather than running all the time.
- After the crowd decreases or the temperature and air quality improve, the system reduces ventilation to save energy.

How it functions in the project: When the mosque becomes crowded, the sensor readings increase and indicate a higher need for ventilation. The controller then activates the fans or AC to improve air circulation and maintain comfort for worshippers. If the temperature, humidity, and CO₂ levels return to acceptable values, the system stops or reduces the output again. This makes the ventilation process both responsive and energy efficient, since it is based on actual indoor conditions rather than fixed manual operations.

9.2.3 Solenoid Valve

Automatically closes water supply during abnormal flow patterns and can reopen via logic/manual reset.

The solenoid valve acts as an output device that controls the water supply in the ablution area based on the readings from the water flow sensor. Its role is to stop water waste when abnormal or continuous flow is detected, such as when a tap is left open or a leak occurs.

Working principle:

- The water flow sensor continuously measures the amount of water moving through the pipe.
- If the system detects abnormal flow or flow that continues longer than expected, the controller interprets this as possible waste.
- The controller then sends an electrical signal to the solenoid valve.
- The solenoid valve closes and cuts off the water supply.

- When normal conditions return, the valve can be opened again manually or by the control logic.

How it functions in the project: When the sensor detects that water is flowing longer than it should, the system assumes that the tap may have been left open or that a leak is present. The controller then activates the solenoid valve to stop the flow and prevent waste. This helps the mosque conserve water while still allowing normal use when water demand is expected.

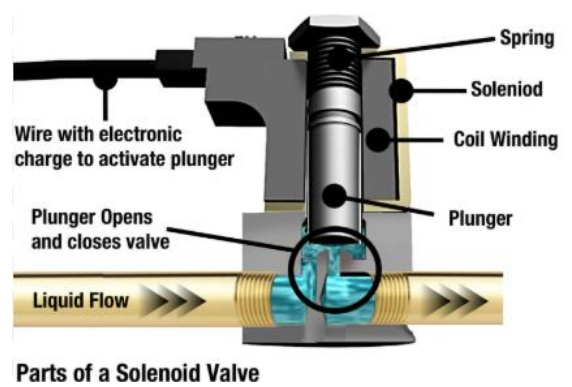


Figure 14: Solenoid valve components

9.3 Processing Unit

The ESP32 is selected for integrated Wi-Fi/Bluetooth, sufficient GPIO and peripheral interfaces (I2C/UART/SPI/PWM), low power operation, and capability to run concurrent IoT control tasks.

The ideal processing unit for this project is the ESP32 microcontroller, as it is a powerful, low-cost, low-power System on a Chip (SoC), and offers enough processing capability to handle multiple sensors, control actuators, and support real-time IoT automation. It is also suitable for this system because it includes built-in Wi-Fi and Bluetooth, has multiple GPIO pins, and supports low-power operation, which makes it practical for a smart mosque environment.

- Wi-Fi connectivity: The ESP32 has integrated Wi-Fi, so it can support network-based automation and monitoring without needing an extra communication module.
- Suitable for multiple inputs and outputs: It provides many GPIO pins and supports interfaces such as I2C, UART, SPI, and PWM, which makes it easy to connect PIR sensors, flow sensors, IAQ sensors, relays, and valves.
- Low power and efficient: The ESP32 includes power-saving modes, which is useful for a system that may run continuously throughout the day.
- Good for embedded IoT control: It is designed for IoT applications and can act as a standalone controller for sensor reading and device actuation.
- Practical for project scope: Compared with simpler boards, the ESP32 is better suited to a connected system that must manage occupancy, ventilation, lighting, and water control at the same time.

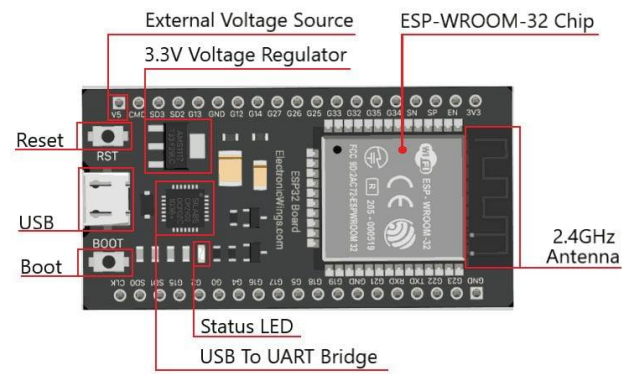


Figure 15: ESP32 processing unit

9.4 Communication Layer

- Local Wi-Fi AP for ESP32-to-server connectivity
- IEEE 802.11 with WPA2-PSK security
- Star topology
- IoT network segmentation from general user devices
- CPT representation: LAN linking MCU units to Registration Server

The communication layer is responsible for enabling reliable, real-time data exchange between distributed sensor nodes, the central processing unit, and the monitoring interface. It ensures that both sensing data and control commands are transmitted efficiently while maintaining system security and stability.

The system communication is implemented as follows:

- Network Setup: A local Wi-Fi access point connects ESP32 nodes to the central server, providing reliable coverage within the mosque.
- Protocol and Security: Communication is based on IEEE 802.11 Wi-Fi and secured using WPA2-PSK encryption to restrict access and protect transmitted data.
- Topology: A star topology is used, where all nodes communicate directly with the central access point, simplifying management and system expansion.
- Network Segmentation: IoT devices are placed on a separate network segment from general user devices, reducing interference and minimizing unauthorized access risks.
- Real-World Implementation: In practical deployments, technologies such as Zigbee or MQTT over Wi-Fi with a local broker may be used for improved scalability and efficiency.
- Simulation Representation: In Cisco Packet Tracer, this is modeled as a local LAN connecting MCU units to the Registration Server, which acts as the central communication hub.

9.5 Integration Workflow

The integration workflow follows the same process defined in the system process specification (Step 2), ensuring consistent interaction between sensing, processing, and actuation components.

1. Sensing stage: Sensors continuously monitor system conditions, including occupancy through PIR sensors, environmental parameters (CO₂, temperature, and humidity), and water usage through flow sensors in the wudu area.
2. Data transmission stage: Sensor readings are transmitted via the local Wi-Fi network to the central processing unit for analysis.
3. Processing and decision stage: The processing unit evaluates incoming data based on predefined conditions such as prayer schedules, occupancy detection, and environmental thresholds. Decisions are made regarding whether system actions are required.
4. Actuation stage: Based on the processed data, control signals are sent to actuators. Lighting and ventilation systems are activated when occupancy is detected during prayer times, ventilation is adjusted in response to environmental conditions, and water flow is controlled to prevent wastage.

5. Monitoring and feedback stage: System status and sensor data are updated on the central dashboard, allowing the administrator to monitor system performance and apply manual overrides when necessary.

This workflow is consistent with Step 2 process design.

9.6 System Diagram

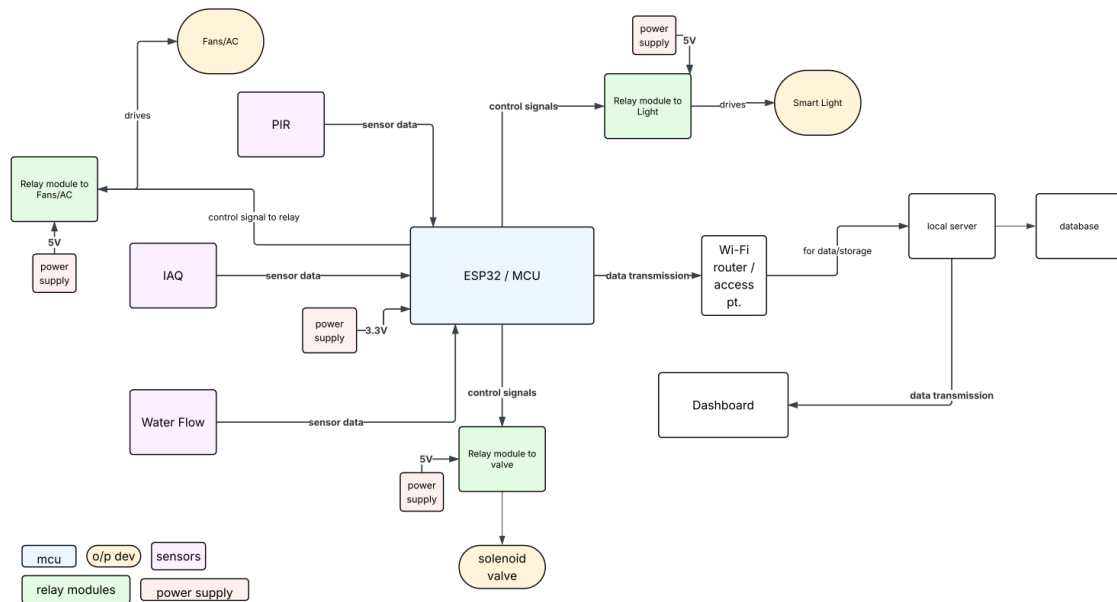


Figure 16: System integration diagram

10. Application Development

10.1 Application Overview and Platform

The application layer of the IoT system consists of a web-based dashboard that serves as the main interface between users and the system. It provides a centralized platform for interacting with real-time data collected from sensors and controlling connected devices.

Its main purpose is to enable effective monitoring and management of the IoT environment by presenting live information in a clear and organized way. The dashboard acts as the primary operational tool for users, allowing them to observe system status, respond to alerts, and manage device behavior efficiently.

The application layer is a web-based dashboard acting as the primary interface between users and the IoT system. It centralizes real-time data visualization and device control in one operational view.

10.2 User Role

The system is designed exclusively for admin users, who are responsible for monitoring and managing the IoT environment through the web-based dashboard. The admin has full access to real-time sensor data, system status, and device controls. Their role includes overseeing environmental conditions, responding to alerts, and adjusting system settings when necessary to ensure optimal performance and safety.

The system is designed for an admin role with full access to monitoring, alert handling, and override control to maintain system performance and safety.

10.3 Core Features

The web-based dashboard provides a set of essential functionalities that enable effective monitoring and control of the IoT system, including the following core features:

- Real-time monitoring of sensor data (e.g., CO₂, temperature, humidity, occupancy)
- Live system status display for all connected devices
- Alert and notification system for abnormal conditions
- Control of IoT actuators (e.g., ventilation system)
- Overview of historical data visualization and basic trends (e.g., peak hours)

The web-based dashboard communicates with the IoT system through a backend layer that enables real-time data exchange and control. It continuously receives updated sensor readings and system statuses, while also sending user commands to connected devices for execution.

Application-system interaction:

- Retrieves real-time data from IoT sensors via backend APIs
- Displays continuously updated system status on the dashboard
- Sends control commands to actuators (e.g., ventilation system)
- Supports bidirectional communication for monitoring and control

- Ensures low-latency updates for real-time system responsiveness

10.4 User Interface Design

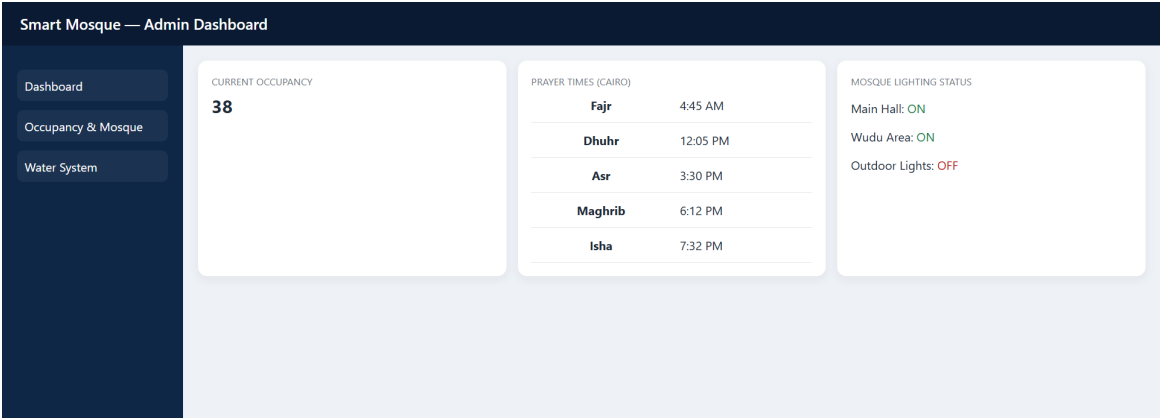


Figure 17: Dashboard UI design - Prayer times

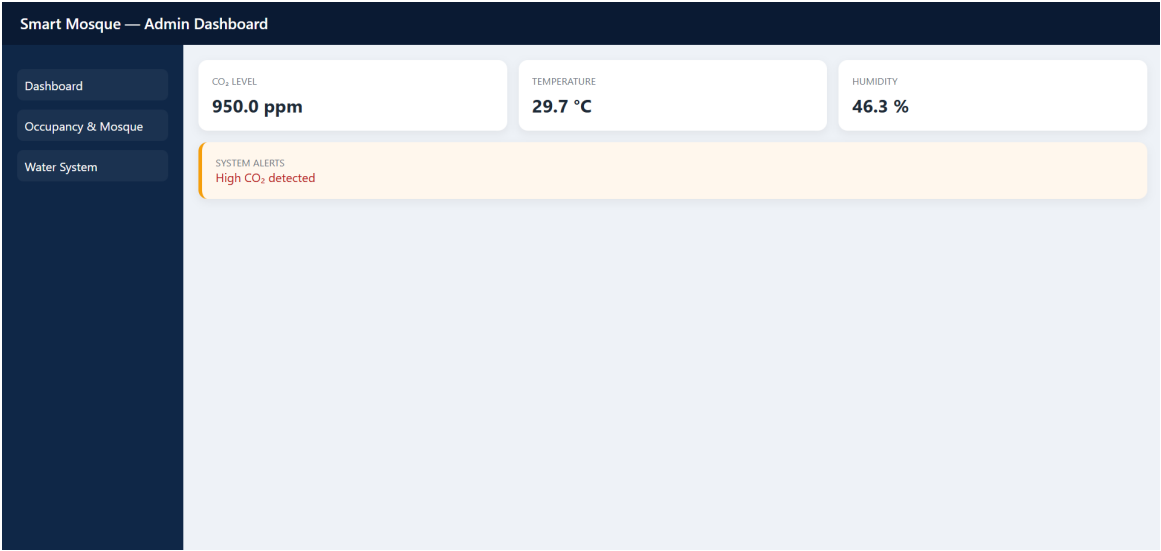


Figure 18: Dashboard UI design - High CO₂ alert

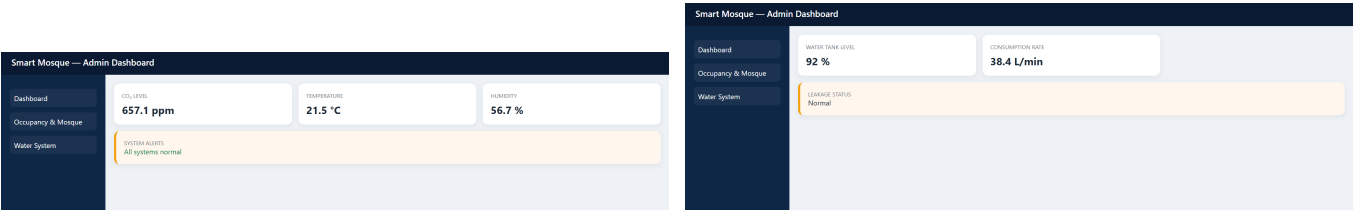


Figure 19: Dashboard UI design - Normal conditions

List of Figures

1	Integrated system flowchart	7
2	UML domain model diagram	10
3	Main data flow	12
4	Main flow legend	12
5	Control flow	12
6	IoT system layer interactions	20
7	Inter-block data flow	23
8	Operational response-time trend	24
9	PIR motion sensor (left) and PIR sensing grid illustration (right)	30
10	Infrared detection concept (left) and PIR sensor structure (right)	31
11	Motion detection behavior	31
12	Indoor air quality sensor module	32
13	Water flow sensor	34
14	Solenoid valve components	36
15	ESP32 processing unit	37
16	System integration diagram	39
17	Dashboard UI design - Prayer times	42
18	Dashboard UI design - High CO ₂ alert	42
19	Dashboard UI design - Normal conditions	42

References

- PIR Motion Sensor: <https://www.electronicwings.com/sensors-modules/pir-sensor>
- IAQ Sensor: <https://www.winsen-sensor.com/knowledge/iaq-sensor.html>
- Water Flow Sensor: <https://www.elprocus.com/a-memoir-on-water-flow-sensor/>
- ESP32 Processing Unit: <https://www.espressif.com/en/products/socs/esp32>
- IoT Response Time: <https://ieeexplore.ieee.org/document/9043102>
- Carbon Dioxide Levels:
<https://www.co2meter.com/blogs/news/carbon-dioxide-indoor-levels-chart>
- Air Quality Monitoring:
<https://www.sciencedirect.com/science/article/pii/S0360132325008686>
- Water Flow Monitoring: <https://new.abb.com/products/measurement-products/measurement-products-blog/water-flowmeters-for-enhanced-leakage-detection>
- IoT Security: <https://www.digi.com/blog/post/iot-security>
- Access Control: <https://thingsboard.io/docs/pe/user-guide/rbac/>
- Data Encryption:
<https://www.buildings.com/safety-security/cybersecurity/article/33017912/trust-no-one-3-steps-to-secure-iot-devices-and-building-it-and-ot-networks>
- WPA2-PSK Background: <https://superops.com/tech-hub/what-is-wpa2-psk>
- Device Filtering: <https://www.portnox.com/cybersecurity-101/network-security/what-is-a-mac-filtering-router/>
- MAC Filtering: <https://blog.scalefusion.com/what-is-mac-filtering/>